

LL(1) Parser Evaluation & design

$$S \rightarrow S+T/T$$

$$T \rightarrow T*F/F$$

$$F \rightarrow (s)/id$$

1. Left Recursion Eliminating

$$S \rightarrow TS'$$

$$S' \rightarrow +TS'/\epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT'/\epsilon$$

$$F \rightarrow (s)/id$$

2. First & follow

$$\text{First}(S) = \{ (, id \}$$

$$\text{First}(S') = \{ +, \epsilon \}$$

$$\text{First}(T) = \{ (, id \}$$

$$\text{First}(T') = \{ *, \epsilon \}$$

$$\text{First}(F) = \{ (, id \}$$

$$\text{Follow}(S) = \{ \$,) \}$$

$$\text{Follow}(S') = \{ \$,) \}$$

$$\text{Follow}(T) = \{ +, \$,) \}$$

Follow (T') = {+, \$}

Follow (F) = {*, +, \$}

LL(1) Parsing table

Non Terminal	id	+	*	(	)	\$
S	$S \rightarrow TS$			$S \rightarrow TS$		
S'		$S' \rightarrow +TS$			$S' \rightarrow \epsilon$	$S' \rightarrow \epsilon$
T	$T \rightarrow FT$			$T \rightarrow FT$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *T$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow (S)$			$F \rightarrow (S)$		

Grammar satisfies LL(1) condition

• No multiple entries: No two productions in the single cell

∴ It satisfies LL(1) condition

Stack Implementation

Stack	Input	Output / Action
\$	id + id * id \$	
\$ sl +	id + id * id \$	S → Tsl
\$ sl +	id + id * id \$	T → Ftl
\$ sl + f	id + id * id \$	F → id
\$ sl + id	id + id * id \$	
\$ sl +	+ id * id \$	Tl → E
\$ sl	+ id * id \$	sl → +sl
\$ sl +	+ id * id \$	
\$ sl +	* id \$	
\$ sl +	* id \$	
\$ sl +	id * id \$	T → Ftl
\$ sl + f	id * id \$	F → id
\$ sl + id	* id \$	
\$ sl +	* id \$	sl → E
\$ +	* id \$	tl → *ftl
\$ tl + f *	* id \$	
\$ tl + f	id \$	F → id
\$ tl + id	id \$	
\$ tl	\$	tl → E
\$	\$	

∴ string Accepted of full trace ∴ Input Accepted

```
# LL(1) Predictive Parser
# Compiler Design Assignment
```

```
grammar = {
    "E": [["T", "E'"]],
    "E'": [["+", "T", "E'"], ["ε"]],
    "T": [["F", "T'"]],
    "T'": [["*", "F", "T'"], ["ε"]],
    "F": [["(", "E", ")"], ["id"]]
}
```

```
FIRST = {
    "E": {"id", "("},
    "E'": {"+", "ε"},
    "T": {"id", "("},
    "T'": {"*", "ε"},
    "F": {"id", "("}
}
```

```
FOLLOW = {
    "E": {"$", ")"},
    "E'": {"$", ")"},
    "T": {"+", "$", ")"},
    "T'": {"+", "$", ")"},
    "F": {"*", "+", "$", ")" }
}
```

```
parsing_table = {
    ("E", "id"): ["T", "E'"],
    ("E", "("): ["T", "E'"],

    ("E'", "+"): ["+", "T", "E'"],
    ("E'", "$"): ["ε"],
    ("E'", ")"): ["ε"],

    ("T", "id"): ["F", "T'"],
    ("T", "("): ["F", "T'"],

    ("T'", "*"): ["*", "F", "T'"],
    ("T'", "+"): ["ε"],
    ("T'", "$"): ["ε"],
    ("T'", ")"): ["ε"],

    ("F", "id"): ["id"],
    ("F", "("): ["(", "E", ")"]
}
```

```
def print_first_follow():
    print("\nFIRST Sets")
```

```

for k, v in FIRST.items():
    print(f"FIRST({k}) = {v}")

print("\nFOLLOW Sets")
for k, v in FOLLOW.items():
    print(f"FOLLOW({k}) = {v}")

def print_table():
    print("\nLL(1) Parsing Table\n")

    headers = ["id", "+", "*", "(", ")", "$"]
    print("{:<8}".format(""), end="")
    for h in headers:
        print("{:<18}".format(h), end="")
    print()

    nts = ["E", "E'", "T", "T'", "F"]

    for nt in nts:
        print("{:<8}".format(nt), end="")
        for h in headers:
            if (nt, h) in parsing_table:
                rhs = " ".join(parsing_table[(nt, h)])
                print("{:<18}".format(rhs), end="")
            else:
                print("{:<18}".format("-"), end="")
        print()

def predictive_parse(tokens):

    stack = ["$", "E"]
    tokens.append("$")

    step = 1

    print("\nParsing Steps\n")
    print("{:<5}{:<30}{:<25}{:}".format(
        "Step", "Stack", "Input", "Action"))

    while stack:

        top = stack[-1]
        current = tokens[0]

        print("{:<5}{:<30}{:<25}".format(
            step,
            " ".join(stack),
            " ".join(tokens)

```

```

    ), end="")

    if top == current == "$":
        print("Accepted")
        return

    elif top == current:
        stack.pop()
        tokens.pop(0)
        print("Match", current)

    elif top in grammar:

        if (top, current) not in parsing_table:
            print("Error")
            return

        production = parsing_table[(top, current)]

        stack.pop()

        if production != ["ε"]:
            for symbol in reversed(production):
                stack.append(symbol)

        print(top, "->", " ".join(production))

    else:
        print("Error")
        return

    step += 1

print("=" * 60)
print("LL(1) Predictive Parser")
print("=" * 60)

print("\nGrammar")

for lhs, rhs in grammar.items():
    for prod in rhs:
        print(lhs, "->", " ".join(prod))

print_first_follow()

print_table()

print("\nInput String : id + id * id")

```

```
tokens = ["id", "+", "id", "*", "id"]
```

```
predictive_parse(tokens)
```

Python 3.12.2 (tags/v3.12.2:6abddd9, Feb 6 2024, 21:26:36) [MSC v.1937 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license()" for more information.

= RESTART: C:/Users/santh/AppData/Local/Programs/Python/Python312/q1 compiler.py

=====

LL(1) Predictive Parser

=====

Grammar

E -> T E'

E' -> + T E'

E' -> ϵ

T -> F T'

T' -> * F T'

T' -> ϵ

F -> (E)

F -> id

FIRST Sets

FIRST(E) = {'(', 'id'}

FIRST(E') = {' ϵ ', '+'}

FIRST(T) = {'(', 'id'}

FIRST(T') = {' ϵ ', '*'}

FIRST(F) = {'(', 'id'}

FOLLOW Sets

FOLLOW(E) = {')', '\$'}

FOLLOW(E') = {')', '\$'}

FOLLOW(T) = {'+', ')', '\$'}

FOLLOW(T') = {'+', ')', '\$'}

FOLLOW(F) = {'+', '\$', ')', '*'}

LL(1) Parsing Table

	id		+	*	(	)
		\$				
E	T E'	-	-	-	T E'	-
E'	-	-	+ T E'	-	-	ϵ
		ϵ				
T	F T'	-	-	-	F T'	-
		-				
T'	-	ϵ	* F T'	-	-	ϵ
		ϵ				
F	id	-	-	-	(E)	-
		-				

Input String : id + id * id

Parsing Steps

Step	Stack	Input	Action
1	\$ E	id + id * id \$	E -> T E'
2	\$ E' T	id + id * id \$	T -> F T'
3	\$ E' T' F	id + id * id \$	F -> id
4	\$ E' T' id	id + id * id \$	Match id
5	\$ E' T'	+ id * id \$	T' -> ϵ
6	\$ E'	+ id * id \$	E' -> + T E'
7	\$ E' T +	+ id * id \$	Match +
8	\$ E' T	id * id \$	T -> F T'
9	\$ E' T' F	id * id \$	F -> id
10	\$ E' T' id	id * id \$	Match id
11	\$ E' T'	* id \$	T' -> * F T'
12	\$ E' T' F *	* id \$	Match *
13	\$ E' T' F	id \$	F -> id
14	\$ E' T' id	id \$	Match id
15	\$ E' T'	\$	T' -> ϵ
16	\$ E'	\$	E' -> ϵ
17	\$	\$	Accepted

```
# Q2 - Shift Reduce Parser
# Compiler Design Assignment
```

```
def print_state(step, stack, input_buffer, action):
    print(f"{step:<5}{' ' * 10}.join(stack):<30}{' ' * 10}.join(input_buffer):<30}{action}")
```

```
tokens = ["id", "+", "id", "*", "id", "$"]
stack = []
step = 1
```

```
print("=" * 85)
print("SHIFT REDUCE PARSER")
print("=" * 85)
print(f"{'Step':<5}{'Stack':<30}{'Input Buffer':<30}Action")
print("-" * 85)
```

```
while True:
```

```
    # Reduce id -> E
    if stack and stack[-1] == "id":
        stack[-1] = "E"
        print_state(step, stack, tokens, "Reduce E -> id")
        step += 1
        continue
```

```
    # Reduce E * E
    if len(stack) >= 3 and stack[-3:] == ["E", "*", "E"]:
        stack = stack[:-3]
        stack.append("E")
        print_state(step, stack, tokens, "Reduce E -> E * E")
        step += 1
        continue
```

```
    # Reduce E + E (only when '*' is not waiting)
    if len(stack) >= 3 and stack[-3:] == ["E", "+", "E"]:
        if "*" not in tokens:
            stack = stack[:-3]
            stack.append("E")
            print_state(step, stack, tokens, "Reduce E -> E + E")
            step += 1
            continue
```

```
    # Accept
    if stack == ["E"] and tokens == ["$"]:
        print_state(step, stack, tokens, "ACCEPT")
        break
```

```
    # Shift
    if tokens:
        symbol = tokens.pop(0)
```

```
    if symbol == "$":
        continue
    stack.append(symbol)
    print_state(step, stack, tokens + ["$"], f"Shift '{symbol}'")
    step += 1
else:
    print("\nERROR: Invalid String")
    break
```

Python 3.12.2 (tags/v3.12.2:6abddd9, Feb 6 2024, 21:26:36) [MSC v.1937 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license()" for more information.

>>>

= RESTART: C:/Users/santh/AppData/Local/Programs/Python/Python312/q2_compiler.py

==

SHIFT REDUCE PARSER

==

Step	Stack	Input Buffer	Action
--			
1	id	+ id * id \$ \$	Shift 'id'
2	E	+ id * id \$	Reduce E -> id
3	E +	id * id \$ \$	Shift '+'
4	E + id	* id \$ \$	Shift 'id'
5	E + E	* id \$	Reduce E -> id
6	E + E *	id \$ \$	Shift '*'
7	E + E * id	\$ \$	Shift 'id'
8	E + E * E	\$	Reduce E -> id
9	E + E	\$	Reduce E -> E * E
10	E	\$	Reduce E -> E + E
11	E	\$	ACCEPT

```

# Q3 - FIRST and FOLLOW Sets
# Grammar:
# S -> L = R | R
# L -> * R | id
# R -> L

grammar = {
    "S": [["L", "=", "R"], ["R"]],
    "L": [["*", "R"], ["id"]],
    "R": [["L"]]
}

non_terminals = set(grammar.keys())
terminals = {"id", "*", "=", "$"}

FIRST = {nt: set() for nt in non_terminals}
FOLLOW = {nt: set() for nt in non_terminals}

FOLLOW["S"].add("$")

def first(symbol):
    if symbol not in grammar:
        return {symbol}

    if FIRST[symbol]:
        return FIRST[symbol]

    result = set()

    for production in grammar[symbol]:
        first_symbol = production[0]

        if first_symbol in grammar:
            result |= first(first_symbol)
        else:
            result.add(first_symbol)

    FIRST[symbol] = result
    return result

# Compute FIRST
for nt in grammar:
    first(nt)

# Compute FOLLOW
changed = True

while changed:

```

```

changed = False

for lhs in grammar:
    for production in grammar[lhs]:

        for i, symbol in enumerate(production):

            if symbol in non_terminals:

                if i + 1 < len(production):

                    next_symbol = production[i + 1]

                    if next_symbol in non_terminals:
                        before = len(FOLLOW[symbol])
                        FOLLOW[symbol] |= FIRST[next_symbol]
                        if len(FOLLOW[symbol]) != before:
                            changed = True
                    else:
                        if next_symbol not in FOLLOW[symbol]:
                            FOLLOW[symbol].add(next_symbol)
                            changed = True

                else:
                    before = len(FOLLOW[symbol])
                    FOLLOW[symbol] |= FOLLOW[lhs]
                    if len(FOLLOW[symbol]) != before:
                        changed = True

print("=" * 50)
print("GRAMMAR")
print("=" * 50)

for lhs in grammar:
    rhs = " | ".join(" ".join(p) for p in grammar[lhs])
    print(f"{lhs} -> {rhs}")

print("\nFIRST Sets")
print("-" * 50)

for nt in FIRST:
    print(f"FIRST({nt}) = {FIRST[nt]}")

print("\nFOLLOW Sets")
print("-" * 50)

for nt in FOLLOW:
    print(f"FOLLOW({nt}) = {FOLLOW[nt]}")

```

Python 3.12.2 (tags/v3.12.2:6abddd9, Feb 6 2024, 21:26:36) [MSC v.1937 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license()" for more information.

>>>

= RESTART: C:\Users\santh\AppData\Local\Programs\Python\Python312\q3 compiler.py

=====

GRAMMAR

=====

S -> L = R | R

L -> * R | id

R -> L

FIRST Sets

FIRST(R) = {'id', '*'}

FIRST(L) = {'id', '*'}

FIRST(S) = {'id', '*'}

FOLLOW Sets

FOLLOW(R) = {'\$', '='}

FOLLOW(L) = {'\$', '='}

FOLLOW(S) = {'\$'}